

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №10.

Тема: «»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

1. Создать пользовательский класс с минимальной функциональностью.
2. Написать функцию для создания объектов пользовательского класса (ввод исходной информации с клавиатуры) и сохранения их в потоке (файле).
3. Написать функцию для чтения и просмотра объектов из потока.
4. Написать функцию для удаления объектов из потока в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
5. Написать функцию для добавления объектов в поток в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
6. Написать функцию для изменения объектов в потоке в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
7. Для вызова функций в основной программе предусмотреть меню.

Анализ задачи

1. Создаём класс Money для хранения денежной суммы в рублях и копейках. Добавляем приватные поля rubles и kopeks.
2. Добавляем приватный метод normalize для нормализации: если копеек больше или равно 100, переводим излишек в рубли; если копеек меньше 0, корректируем рубли и копейки.
3. Создаём конструкторы: по умолчанию (0 рублей, 0 копеек), с параметрами (с вызовом normalize), копирования.
4. Перегружаем операторы: присваивания =, сравнения == и !=, сравнения > и <.
5. Перегружаем арифметические операторы: + для сложения двух сумм, + для сложения суммы с целым числом копеек, += для сложения с присваиванием, - для вычитания двух сумм, -= для вычитания с присваиванием.
6. Перегружаем операторы декремента: постфиксный -- (возвращает старое значение, затем вычитает 1 копейку) и префиксный -- (вычитает 1 копейку, возвращает новое значение).
7. Перегружаем операторы ввода >> и вывода << через дружественные функции (ввод запрашивает рубли и копейки отдельно, вывод форматирует с ведущим нулём для копеек).
8. Создаём функцию inputMoney для ввода массива денежных сумм (запрашиваем количество записей, затем каждую запись через cin).
9. Создаём функцию printMoney для вывода всех записей массива.

10. Создаём функцию `deleteRecords` для удаления записей, превышающих заданное граничное значение. Проходим по массиву, копируем элементы, не превышающие лимит, в начало массива, уменьшаем размер.

11. Создаём функцию `increaseRecords` для увеличения записей с заданным значением на 1 рубль 50 копеек. Ищем совпадения с целевым значением, при нахождении прибавляем `Money(1,50)`.

12. Создаём функцию `addRecords` для добавления `K` записей после записи с номером `N`. Проверяем корректность `N` и достаточно ли места, сдвигаем элементы вправо, вводим новые записи.

13. В функции `main`: создаём статический массив `Money` фиксированного размера `MAX_SIZE=100` и переменную `n` для текущего количества записей. Организуем цикл с меню, где пользователь может выбрать создание записей, просмотр, удаление, изменение, добавление записей или демонстрацию операций --. Программа работает до выбора пункта 0.

Код

```
#include <iostream>
#include <locale>

using namespace std;

class Money {
private:
    long rubles;
    int kopeks;

    void normalize() {
        if (kopeks >= 100) {
            rubles += kopeks / 100;
            kopeks %= 100;
        }
        else if (kopeks < 0) {
            long newRubles = rubles + (kopeks / 100) - 1;
            int newKopeks = 100 + (kopeks % 100);
            if (newKopeks >= 100) {
                newRubles += newKopeks / 100;
                newKopeks %= 100;
            }
            rubles = newRubles;
            kopeks = newKopeks;
        }
    }

public:
    Money() : rubles(0), kopeks(0) {}

    Money(long r, int k) : rubles(r), kopeks(k) {
        normalize();
    }

    Money(const Money& other) : rubles(other.rubles), kopeks(other.kopeks) {}

    ~Money() {}

    Money& operator=(const Money& other) {
        if (this != &other) {
            rubles = other.rubles;

```

```

        kopeks = other.kopeks;
    }
    return *this;
}

bool operator==(const Money& other) const {
    return (rubles == other.rubles && kopeks == other.kopeks);
}

bool operator!=(const Money& other) const {
    return !(*this == other);
}

bool operator>(const Money& other) const {
    if (rubles != other.rubles) {
        return rubles > other.rubles;
    }
    return kopeks > other.kopeks;
}

bool operator<(const Money& other) const {
    if (rubles != other.rubles) {
        return rubles < other.rubles;
    }
    return kopeks < other.kopeks;
}

Money operator+(const Money& other) const {
    Money result;
    result.rubles = rubles + other.rubles;
    result.kopeks = kopeks + other.kopeks;
    result.normalize();
    return result;
}

Money operator+(int kop) const {
    Money result;
    result.rubles = rubles;
    result.kopeks = kopeks + kop;
    result.normalize();
    return result;
}

Money& operator+=(const Money& other) {
    rubles += other.rubles;
    kopeks += other.kopeks;
    normalize();
    return *this;
}

Money operator-(const Money& other) const {
    Money result;
    long total1 = rubles * 100 + kopeks;
    long total2 = other.rubles * 100 + other.kopeks;
    long diff = total1 - total2;
    result.rubles = diff / 100;
    result.kopeks = diff % 100;
    if (result.kopeks < 0) {
        result.rubles--;
        result.kopeks += 100;
    }
    return result;
}

Money& operator--=(const Money& other) {
    long total1 = rubles * 100 + kopeks;
    long total2 = other.rubles * 100 + other.kopeks;
    long diff = total1 - total2;
    rubles = diff / 100;

```

```

        kopeks = diff % 100;
        if (kopeks < 0) {
            rubles--;
            kopeks += 100;
        }
        return *this;
    }

    Money operator--(int) {
        Money temp = *this;
        kopeks -= 1;
        normalize();
        return temp;
    }

    Money& operator--() {
        kopeks -= 1;
        normalize();
        return *this;
    }

    friend istream& operator>>(istream& in, Money& m);
    friend ostream& operator<<(ostream& out, const Money& m);
};

istream& operator>>(istream& in, Money& m) {
    cout << "Введите рубли: ";
    in >> m.rubles;
    cout << "Введите копейки: ";
    in >> m.kopeks;
    m.normalize();
    return in;
}

ostream& operator<<(ostream& out, const Money& m) {
    out << m.rubles << ",";
    if (m.kopeks < 10) {
        out << "0";
    }
    out << m.kopeks;
    return out;
}

void inputMoney(Money* arr, int& n) {
    cout << "Введите количество записей: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "\nЗапись " << i + 1 << ":" << endl;
        cin >> arr[i];
    }
}

void printMoney(Money* arr, int n) {
    if (n == 0) {
        cout << "Нет записей!" << endl;
        return;
    }
    cout << "\nСодержимое:" << endl;
    for (int i = 0; i < n; i++) {
        cout << i + 1 << ". " << arr[i] << endl;
    }
}

void deleteRecords(Money* arr, int& n) {
    if (n == 0) {
        cout << "Нет записей для удаления!" << endl;
        return;
    }
}

```

```

printMoney(arr, n);

Money limit;
cout << "\nВведите граничное значение: ";
cin >> limit;

int newSize = 0;
for (int i = 0; i < n; i++) {
    if (!(arr[i] > limit)) {
        if (newSize != i) {
            arr[newSize] = arr[i];
        }
        newSize++;
    }
    else {
        cout << "Удалена запись: " << arr[i] << " (больше " << limit << ")" << endl;
    }
}

n = newSize;
cout << "Удаление завершено. Осталось записей: " << n << endl;
}

void increaseRecords(Money* arr, int n) {
    if (n == 0) {
        cout << "Нет записей для изменения!" << endl;
        return;
    }

    printMoney(arr, n);

    Money target;
    cout << "\nВведите значение для поиска: ";
    cin >> target;

    Money increment(1, 50);
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            arr[i] += increment;
            cout << "Увеличена запись " << i + 1 << ": " << arr[i] << endl;
            count++;
        }
    }

    if (count == 0) {
        cout << "Записи с таким значением не найдены!" << endl;
    }
    else {
        cout << "Изменение завершено. Изменено записей: " << count << endl;
    }
}

void addRecords(Money* arr, int& n, int maxSize) {
    if (n == 0) {
        cout << "Нет записей! Сначала создайте записи." << endl;
        return;
    }

    printMoney(arr, n);

    int N, K;
    cout << "\nВведите номер записи N (после которой добавлять): ";
    cin >> N;
    cout << "Введите количество записей K для добавления: ";
    cin >> K;

```

```

    if (N < 1 || N > n) {
        cout << "Неверный номер записи!" << endl;
        return;
    }

    if (n + K > maxSize) {
        cout << "Недостаточно места! Максимум " << maxSize << " записей." << endl;
        return;
    }

    for (int i = n - 1; i >= N; i--) {
        arr[i + K] = arr[i];
    }

    for (int i = 0; i < K; i++) {
        cout << "\nНовая запись " << i + 1 << ":" << endl;
        cin >> arr[N + i];
    }

    n += K;
    cout << "Добавление завершено. Всего записей: " << n << endl;
}

int main() {
    setlocale(LC_ALL, "Russian");

    const int MAX_SIZE = 100;
    Money arr[MAX_SIZE];
    int n = 0;
    int choice;

    do {
        cout << "\nМЕНЮ" << endl;
        cout << "1. Создать записи (ввод)" << endl;
        cout << "2. Просмотреть записи" << endl;
        cout << "3. Удалить записи больше заданного значения" << endl;
        cout << "4. Увеличить записи с заданным значением на 1 рубль 50 копеек" << endl;
        cout << "5. Добавить K записей после записи с номером N" << endl;
        cout << "6. Демонстрация операций -- (вычитание копеек)" << endl;
        cout << "0. Выход" << endl;
        cout << "Ваш выбор: ";
        cin >> choice;

        switch (choice) {
            case 1:
                inputMoney(arr, n);
                break;
            case 2:
                printMoney(arr, n);
                break;
            case 3:
                deleteRecords(arr, n);
                break;
            case 4:
                increaseRecords(arr, n);
                break;
            case 5:
                addRecords(arr, n, MAX_SIZE);
                break;
            case 6:
                cout << "\nДемонстрация операций -- (вычитание копеек):" << endl;
                {
                    Money test(10, 50);
                    cout << "Исходная сумма: " << test << endl;

                    Money test2 = test--;
                    cout << "Постфиксный --: вернул " << test2 << ", стало " << test <<
endl;
                }
            default:
                break;
        }
    } while (choice != 0);
}

```

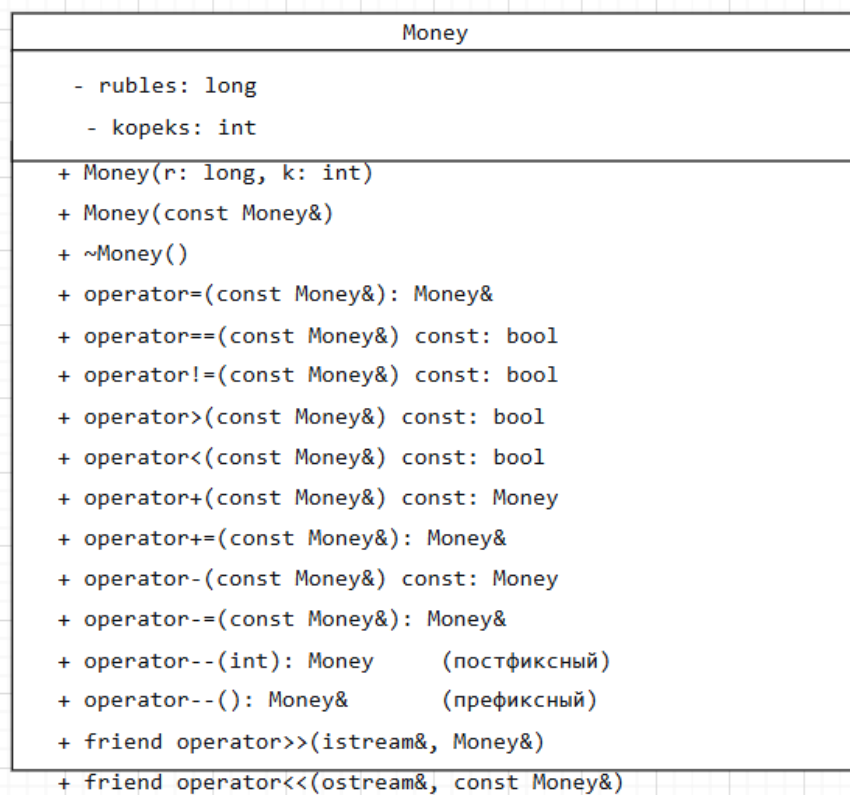
```

    Money test3(5, 20);
    cout << "\nИсходная сумма: " << test3 << endl;
    --test3;
    cout << "Префиксный --: стало " << test3 << endl;
}
break;
case 0:
    cout << "Выход из программы..." << endl;
    break;
default:
    cout << "Неверный выбор!" << endl;
}
} while (choice != 0);

return 0;
}

```

UML диаграмма



Вывод программы

```
МЕНЮ
1. Создать записи (ввод)
2. Просмотреть записи
3. Удалить записи больше заданного значения
4. Увеличить записи с заданным значением на 1 рубль 50 копеек
5. Добавить K записей после записи с номером N
6. Демонстрация операций -- (вычитание копеек)
0. Выход
Ваш выбор: 2
Нет записей!

МЕНЮ
1. Создать записи (ввод)
2. Просмотреть записи
3. Удалить записи больше заданного значения
4. Увеличить записи с заданным значением на 1 рубль 50 копеек
5. Добавить K записей после записи с номером N
6. Демонстрация операций -- (вычитание копеек)
0. Выход
Ваш выбор: 1
Введите количество записей:
3

Запись 1:
Введите рубли: 100
Введите копейки: 50

Запись 2:
Введите рубли: 50
Введите копейки: 30

Запись 3:
Введите рубли: 200
Введите копейки: 75

МЕНЮ
1. Создать записи (ввод)
2. Просмотреть записи
3. Удалить записи больше заданного значения
4. Увеличить записи с заданным значением на 1 рубль 50 копеек
5. Добавить K записей после записи с номером N
6. Демонстрация операций -- (вычитание копеек)
0. Выход
Ваш выбор: 2

Содержимое:
1. 100,50
2. 50,30
3. 200,75

МЕНЮ
1. Создать записи (ввод)
2. Просмотреть записи
3. Удалить записи больше заданного значения
4. Увеличить записи с заданным значением на 1 рубль 50 копеек
5. Добавить K записей после записи с номером N
6. Демонстрация операций -- (вычитание копеек)
0. Выход
Ваш выбор: 30
Неверный выбор!

МЕНЮ
1. Создать записи (ввод)
2. Просмотреть записи
3. Удалить записи больше заданного значения
4. Увеличить записи с заданным значением на 1 рубль 50 копеек
5. Добавить K записей после записи с номером N
6. Демонстрация операций -- (вычитание копеек)
0. Выход
Ваш выбор: 3

Содержимое:
1. 100,50
2. 50,30
3. 200,75

Введите граничное значение: Введите рубли: 100
Введите копейки: 50
Удалена запись: 200,75 (больше 100,50)
Удаление завершено. Осталось записей: 2

МЕНЮ
1. Создать записи (ввод)
2. Просмотреть записи
3. Удалить записи больше заданного значения
4. Увеличить записи с заданным значением на 1 рубль 50 копеек
5. Добавить K записей после записи с номером N
6. Демонстрация операций -- (вычитание копеек)
0. Выход
Ваш выбор: 4
```

Ответы на вопросы

1. Поток — это абстракция последовательности байтов для ввода-вывода данных. Поток связывает программу с источником или приёмником данных (консоль, файл, строка).
2. Типы потоков: входные (istream) для чтения, выходные (ostream) для записи, двунаправленные (iostream) для чтения и записи. Конкретные виды: стандартные, файловые, строковые.
3. Для использования стандартных потоков (cin, cout, cerr, clog) нужно подключить библиотеку <iostream>.
4. Для использования файловых потоков (ifstream, ofstream, fstream) нужно подключить библиотеку <fstream>.
5. Для использования строковых потоков (istringstream, ostringstream, stringstream) нужно подключить библиотеку <sstream>.
6. При выводе в форматированный поток используется операция вставки <<.
7. При выводе из форматированных потоков (чтении) используется операция извлечения >>.
8. Методы для вывода в форматированный поток: put (вывод одного символа), write (вывод блока байтов), а также operator<<.

9. Методы для чтения из форматированного потока: `get` (чтение одного символа), `getline` (чтение строки), `read` (чтение блока байтов), `gcount` (количество прочитанных байтов), а также `operator>>`.

10. Режимы открытия файловых потоков: `ios::in` (чтение), `ios::out` (запись), `ios::app` (добавление в конец), `ios::ate` (переход в конец при открытии), `ios::trunc` (удалить содержимое), `ios::binary` (двоичный режим).

11. Режим для добавления записей в файл — `ios::app`.

12. Конструктор `ifstream file("f.txt")` использует режим `ios::in` (только для чтения).

13. Конструктор `fstream file("f.txt")` использует комбинацию `ios::in | ios::out`. Если файл не существует, он не создаётся.

14. Конструктор `ofstream file("f.txt")` использует режим `ios::out` (только для записи, существующий файл очищается).

15. Режим `ios::out | ios::app` открывает файл для записи с добавлением в конец. Указатель записи всегда в конце файла, содержимое не удаляется.

16. Режим `ios::out | ios::trunc` открывает файл для записи с очисткой существующего содержимого (поведение по умолчанию для `ofstream`).

17. Режим `ios::out | ios::in | ios::trunc` открывает файл для чтения и записи с очисткой содержимого. (Вероятно, опечатка: `trunk` вместо `trunc`).

18. Открыть файл для чтения: `ifstream file("имя.txt");` или `fstream file("имя.txt", ios::in);`.

19. Открыть файл для записи: `ofstream file("имя.txt");` или `fstream file("имя.txt", ios::out);`.

20. Примеры открытия файловых потоков в различных режимах:

```
ifstream f1("file.txt");
ofstream f2("file.txt");
ofstream f3("file.txt", ios::app);
fstream f4("file.txt", ios::in | ios::out);
fstream f5("file.txt", ios::out | ios::trunc);
ifstream f6("file.txt", ios::binary);
```

21. Примеры чтения объектов из потока:

```
ifstream f("data.txt");
int x;
f >> x;
char s[100];
f.getline(s, 100);
double d;
f.read((char*)&d, sizeof(d));
```

22. Примеры записи объектов в поток:

```
ofstream f("data.txt");
f << 10 << " " << 3.14;
int a[5] = {1,2,3,4,5};
f.write((char*)a, sizeof(a));
```

23. Алгоритм удаления записей из файла:

- Открыть исходный файл для чтения.
- Открыть временный файл для записи.
- Читать записи из исходного файла.
- Если запись не подлежит удалению, записать её во временный файл.
- Закрыть оба файла.
- Удалить исходный файл.
- Переименовать временный файл в имя исходного.

24. Алгоритм добавления записей в файл:

- Открыть файл в режиме добавления (ios::app).
- Установить указатель записи в конец файла (автоматически при ios::app).
- Записать новые данные в файл.
- Закрыть файл.

25. Алгоритм изменения записей в файле:

- Открыть исходный файл для чтения.
- Открыть временный файл для записи.
- Читать записи из исходного файла.
- Если запись требует изменения, изменить её перед записью во временный файл.
- Если не требует, записать без изменений.
- Закрыть оба файла.
- Удалить исходный файл.
- Переименовать временный файл в имя исходного файла.